Karnaugh Map

2023/10/16

B113040045 許育菖

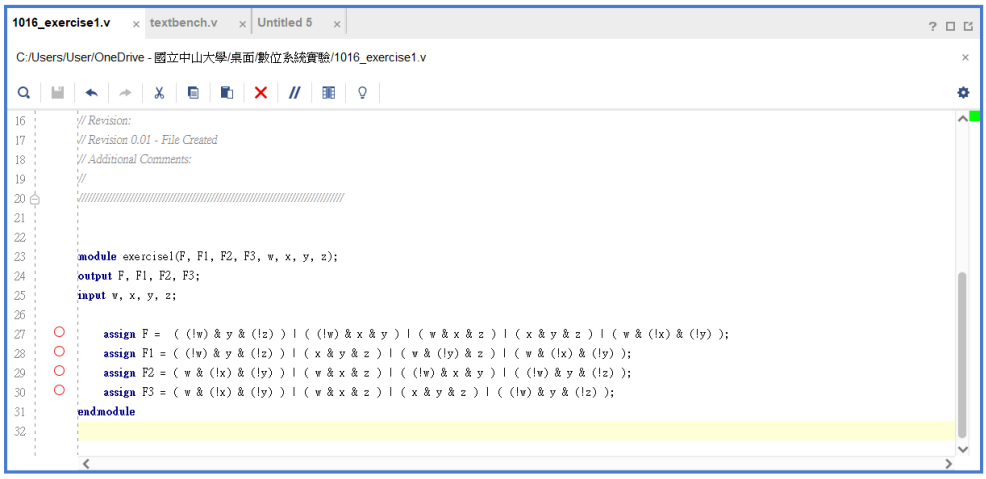
實驗內容

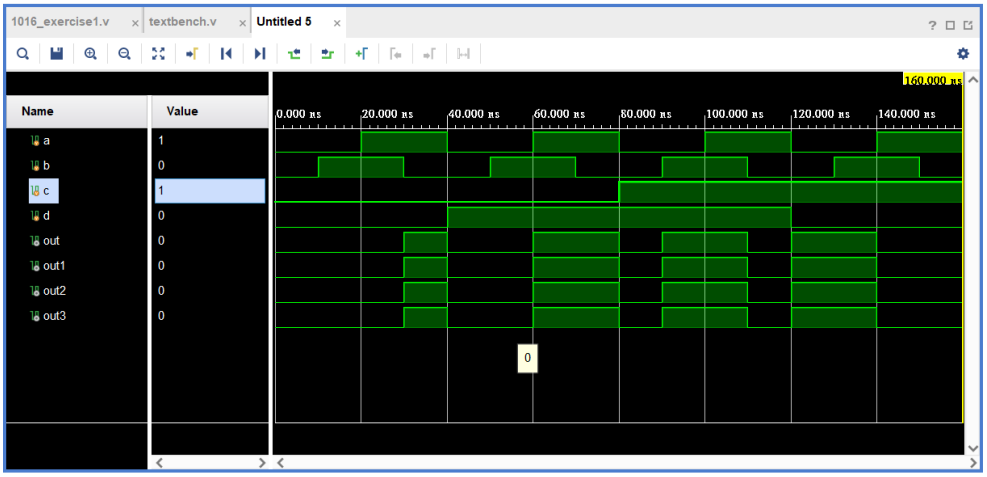
Eercise1

Let F(w, x, y, z) = w'yz' + w'xy + wxz + xyz + wx'y'.

Draw the **Karnaugh map** and find all the simplest sum-of-products of F.

If the simplest sum-of-products of F are F1, F2 and F3, verify F1 = F2 = F3 by using **Verilog Dataflow modeling**.





根據題目，可以得到以下的 Karnaugh Map：

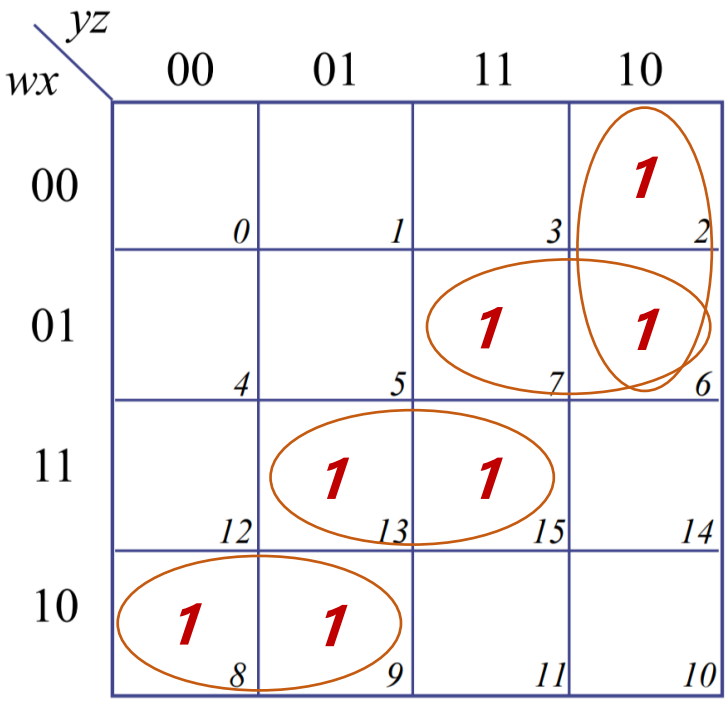
wx \ yz	00	01	11	10
00	0	1	3	1 2
01	4	5	1 7	1 6
11	12	1 13	1 15	14
10	1 8	1 9	11	10

可以發現需要**兩格兩格一組**來得出 sum-of-products。

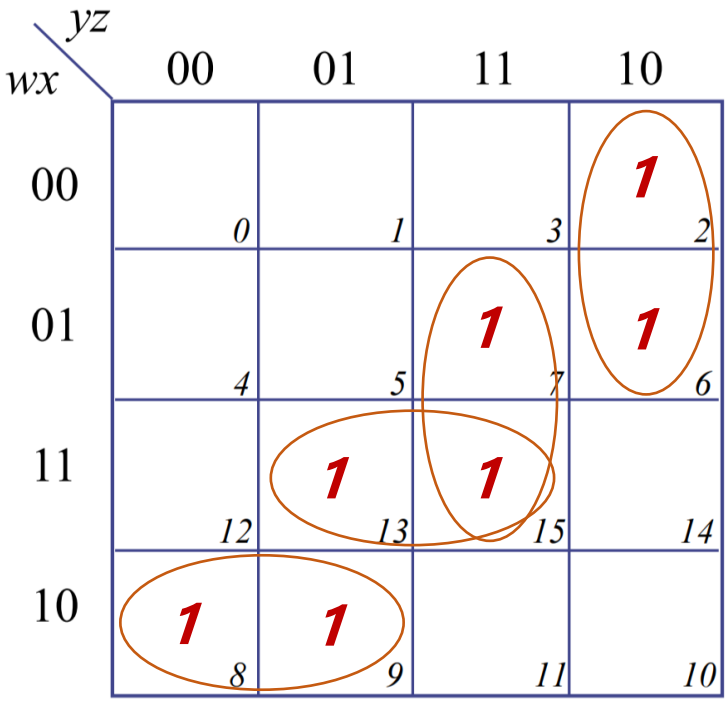
Let F1 = w'yz' + xyz + wy'z + wx'y'

wx \ yz	00	01	11	10
00	0	1	3	1 2
01	4	5	1 7	1 6
11	12	1 13	1 15	14
10	1 8	1 9	11	10

F2 = wx'y' + wxz + w'xy + w'yz'



F3 = wx'y' + wxz + xyz + w'yz'



由實驗結果可以發現：**F(w, x, y, z) = F1 = F2 = F3**

Exercise2

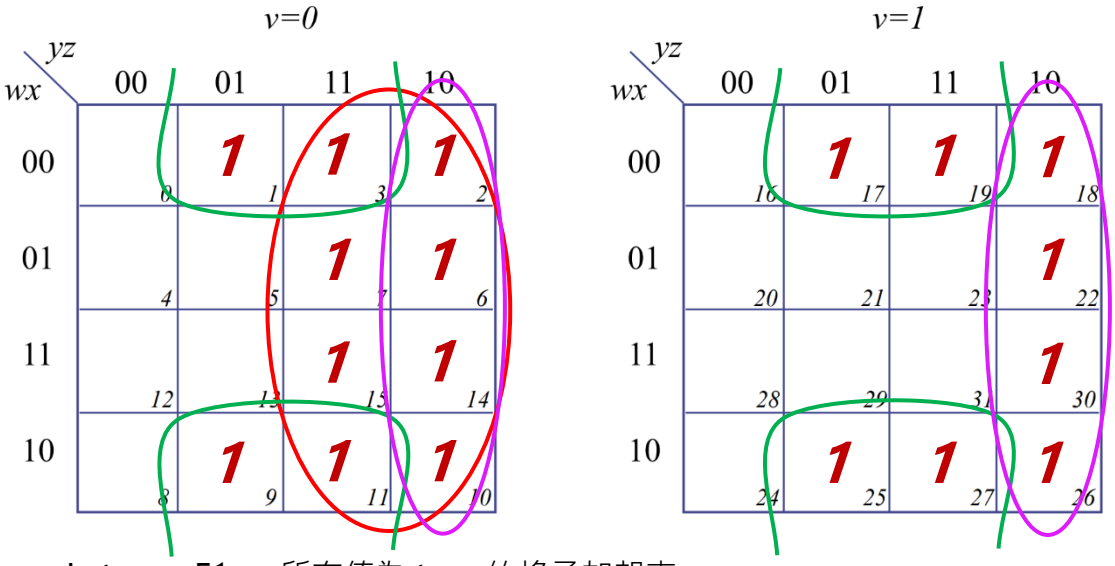
Draw the **Karnaugh map** of F1(v, w, x, y, z) = ∑(1, 2, 3, 6, 7, 9, 10, 11, 14, 15, 17, 18, 19, 22, 25, 26, 27, 30), and find **the simplest sum-of-products** of this Boolean function.

If the simplest sum-of-products of F1 is F2, verify F1 = F2 by using **Verilog Structural level modeling**.

Verilog code：

```
`timescale 1ns / 1ps
module exercise2(v, w, x, y, z, F1, F2);
output F1, F2;
input v, w, x, y, z ;

wire and1, and2, and3, and4, and5, and6, and7, and8, and9, and10;
and(and1, !v, !w, !x, !y, z);
and(and2, !v, !w, !x, y, z);
and(and3, !v, !w, !x, y, !z);

and(and4, !v, !w, x, y, z);
and(and5, !v, !w, x, y, !z);

and(and6, !v, w, x, y, z);
and(and7, !v, w, x, y, !z);

and(and8, !v, w, !x, !y, z);
and(and9, !v, w, !x, y, z);
and(and10, !v, w, !x, y, !z);

wire and11, and12, and13, and14, and15, and16, and17, and18;
and(and11, v, !w, !x, !y, z);
and(and12, v, !w, !x, y, z);
```

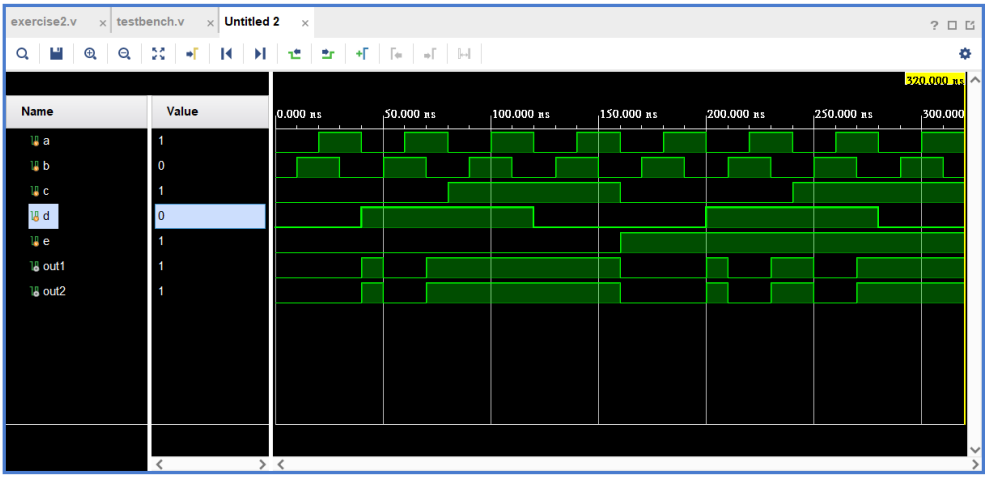
```
and(and13, v, !w, !x, y, !z);

and(and14, v, !w, x, y, !z);

and(and15, v, w, x, y, !z);

and(and16, v, w, !x, !y, z);
and(and17, v, w, !x, y, z);
and(and18, v, w, !x, y, !z);

or(F1, and1, and2, and3, and4, and5, and6, and7, and8, and9, and10, and11, and12, and13,
and14, and15, and16, and17, and18);

wire n1, n2, n3;
and(n1, !x, z);
and(n2, !v, y);
and(n3, y, !z);
or(F2, n1, n2, n3);
endmodule
```



Let F1 = 所有值為 true 的格子加起來

將 Karnaugh map 中值為 true 的格子八個八個一組可得到 F1 的 sum-of-product，把得到的方程式定義為 F2

$\Longrightarrow$ F2 = x'z + v'y + yz'



由實驗結果可發現 **F1 = F2**

Exercise3

Digits		Nine's Complements	
Decimal	BCD	Decimal	BCD
	$x_3\,x_2\,x_1\,x_0$		$y_3\,y_2\,y_1\,y_0$
0	0 0 0 0	9	1 0 0 1
1	0 0 0 1	8	1 0 0 0
2	0 0 1 0	7	0 1 1 1
3	0 0 1 1	6	0 1 1 0
4	0 1 0 0	5	0 1 0 1
5	0 1 0 1	4	0 1 0 0
6	0 1 1 0	3	0 0 1 1
7	0 1 1 1	2	0 0 1 0
8	1 0 0 0	1	0 0 0 1
9	1 0 0 1	0	0 0 0 0

Nine's-complement table

Design the nine's complement converter using Karnaugh maps and don't-care conditions.

Draw the logic diagram of the nine's complement converter and verify the circuit by using **Verilog Structural level modeling**.

Verilog code：

```
`timescale 1ns / 1ps
module exercise3(x0, x1, x2, x3, y0, y1, y2, y3);
        output y0, y1, y2, y3;
        input x0, x1, x2, x3;
//   y0
not(y0, x0);

//   y1
and(y1, 1, x1);

//   y2
wire and1, and2, xor1, xor2;
and(and1, x1, y1);
and(xor1, !x2, and1);

and(and2, !x1, !y1);
and(xor2, x2, and2);
or(y2, xor1, xor2);

//   y3
and(y3, !x3, !x2, !x1);

endmodule
```

題目要求輸入一個變數，然後輸出它的 9 補數，意即該數與它的 9 補數相加起來會等於 9，以 2 進制表示為 1 0 0 1。

從 y_0 往 y_3 做處理，並觀察 Nine’s-complement table 中的規律

我們可以發現：

$y_0 = (x_0)'$

$y_1 = x_1$

而 y_2 比較不好直接觀察出它的規律，但經過計算後可以得到：

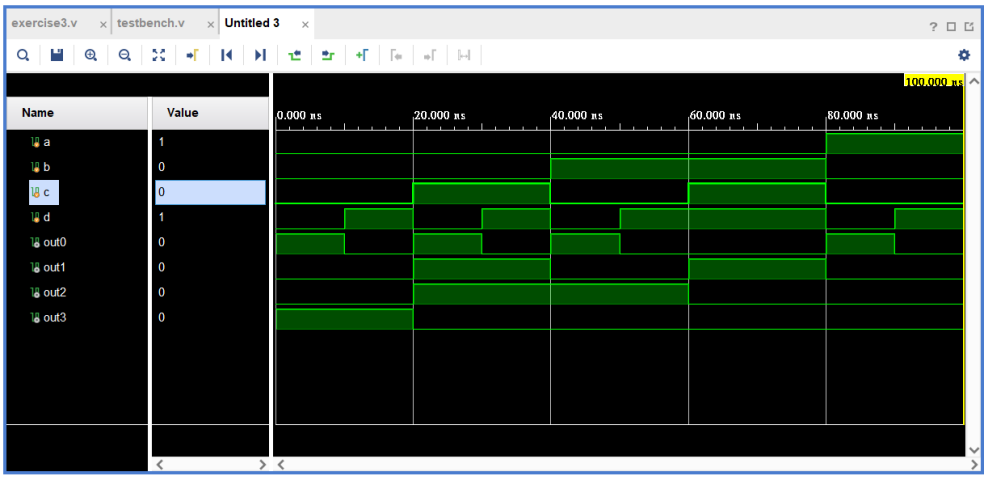
當　$x_1 = y_1 = 1$ and $x_2 = 0$

或是

$x_1 = y_1 = 0$ and $x_2 = 1$ 時，$y_2 = 1$。

其餘情況 $y_2 = 0$。

最後 y_3 則是當 $x_2, x_1, x_0 = 0$ 時，$y_3 = 1$，否則 $y_3 = 0$。



實驗心得

這次實驗室我們第一次接觸 Karnaugh Map，老實說一開始完全不知道如何下手，還去重翻了之前上課的 PPT，才總算有點頭緒。在實驗過程中，我對 Karnaugh Map 有點改觀，原本我以為它是一個麻煩的東西，沒想到使用它來找出 sum-of-product 還滿方便的，尤其是在知道原本的解的時候。不過這次的第二題好麻煩喔，看完題目發現至少要寫 18 個 node，當下真的還滿心累的。